Bridge

dge · Draw shapes with Vector vs Raster renderers independently · Build with Wa

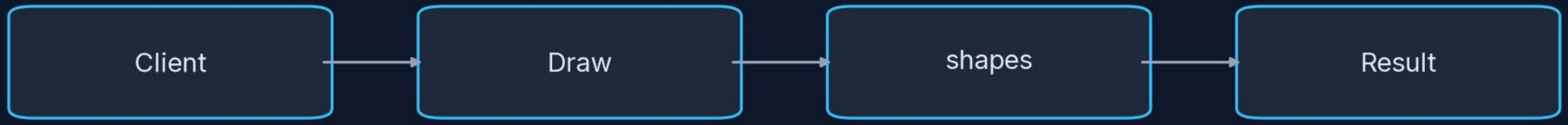
Today's Hook

- Today's force: Draw shapes with Vector vs Raster renderers independently
- We earn Bridge by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Renderer, VectorRenderer, RasterRenderer, Shape

The Problem

- Scenario: Draw shapes with Vector vs Raster renderers independently
- Constraints: solve «Draw shapes with Vector vs Raster renderers independently» with Bridge — keep the demo runnable and the failure path observable.
- Naive pain: explosion of combinations
- Success looks like: vector-circle r=3

Structure



Bridge: Draw shapes with Vector vs Raster renderers indepe

Real-World Scenario

- Running example: Draw shapes with Vector vs Raster renderers independently
- Key types: Renderer, VectorRenderer, RasterRenderer, Shape
- Happy-path stdout cue: vector-circle r=3
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

public final class NaiveShapes {
  public static String vectorCircle(int r){ return "vector-circle r="+r; }
  public static String rasterCircle(int r){ return "raster-circle r="+r; }
  public static String vectorSquare(int s){ return "vector-square s="+s; }
  // explosion of combinations
}
```

Why Naive Hurts

- Explosion of combinations
- Without Bridge, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

interface Renderer { String draw(String shape); }
record VectorRenderer() implements Renderer { public String draw(String s){ return "vector-"+s; } }
record RasterRenderer() implements Renderer { public String draw(String s){ return "raster-"+s; } }
abstract class Shape {
  protected final Renderer r; protected Shape(Renderer r){ this.r=r; }
  public abstract String draw();
}
final class Circle extends Shape {
  private final int radius; public Circle(Renderer r, int radius){ super(r); this.radius=radius; }
  public String draw(){ return r.draw("circle r="+radius); }
}
final class Square extends Shape {
  private final int side; public Square(Renderer r, int side){ super(r); this.side=side; }
  public String draw(){ return r.draw("square s="+side); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Bridge: shape × renderer ===");
    System.out.println("--- naive ---");
System.out.println(NaiveShapes.vectorCircle(3));
System.out.println("--- bridge ---");
System.out.println(new Circle(new VectorRenderer(), 3).draw());
System.out.println(new Square(new RasterRenderer(), 4).draw());
  }
}
```

Walkthrough Output

```
=== Bridge: shape × renderer ===
--- naive ---
vector-circle r=3
--- bridge ---
vector-circle r=3
raster-square s=4
```

Step-by-Step Trace

- Match stdout to stages — look for: vector-circle r=3
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 09-bridge ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

interface Renderer { String draw(String shape); }
record VectorRenderer() implements Renderer { public String draw(String s){ return "vector-"+s; } }
record RasterRenderer() implements Renderer { public String draw(String s){ return "raster-"+s; } }
abstract class Shape {
  protected final Renderer r; protected Shape(Renderer r){ this.r=r; }
  public abstract String draw();
}
final class Circle extends Shape {
  private final int radius; public Circle(Renderer r, int radius){ super(r); this.radius=radius; }
  public String draw(){ return r.draw("circle r="+radius); }
}
final class Square extends Shape {
  private final int side; public Square(Renderer r, int side){ super(r); this.side=side; }
  public String draw(){ return r.draw("square s="+side); }
}
```

Trade-offs

- Use Bridge when «Draw shapes with Vector vs Raster renderers independently» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Bridge in product language.

Related Patterns

- Name the pattern usually confused with Bridge and separate the forces.
- Sketch the roles for «Draw shapes with Vector vs Raster renderers independently» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Draw shapes with Vector vs Raster renderers independently
- Naive pain → Bridge boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.